

Poker Hand Modeling: A Simple Guide

Your modeling process

1. Baseline model

Always guesses the most common hand, high card. This gives you a simple reference score.

2. Decision tree

Learns rules from the cards to predict the hand type.

3. Random forest

Uses many decision trees and combines their predictions.

4. Compare their performance

Evaluate all three on the same validation data using accuracy, macro F1-score, and other useful metrics.

Let's break this down using your poker hand project and keep it simple.

Your goal is to build a machine learning model that looks at five cards and predicts the type of poker hand, such as a pair, a flush, or a full house.

Before you can tell whether your model is good, you need a simple model to compare it against. That's your baseline model.

1. Build the baseline model

Think of the baseline as your starting point. It's a simple prediction method that gives you a reference score.

For your poker dataset, your simplest baseline could be to always predict the most common poker hand category.

From your data, the most common category is **high card**, which makes up about **50.12%** of the hands.

So, if your model predicts "high card" for every single hand, its accuracy would be about 50.12% on the combined data used for your descriptive analysis.

Why is this useful? Imagine you train a decision tree and it achieves 70% accuracy. You can compare it to your baseline of about 50.12% and see that the decision tree is doing better than a very simple prediction strategy.

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score

X = df.drop(columns=["CLASS"])
y = df["CLASS"]

baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X, y)
y_pred = baseline.predict(X)
print(accuracy_score(y, y_pred))
```

This code illustrates the idea, but for a fair model comparison, evaluate the baseline on the same validation data as your other models, rather than calculating its score on the data it was trained on.

2. Train your candidate models

A candidate model is a machine learning model you want to test to see how well it predicts poker hands.

Model 1: Decision tree

A decision tree makes predictions by following a series of questions, such as whether all five cards have the same suit, whether ranks are consecutive, or whether any ranks match. Based on the answers, it follows branches until it predicts a poker hand category. It's a good starting model because it's relatively easy to understand and explain.

Model 2: Random forest

A random forest combines predictions from many decision trees. Instead of relying on one tree, it uses multiple trees and combines their predictions through voting. This can help it capture more complex patterns and reduce the tendency of an individual tree to overfit.

How to train both models in Python, assuming you've already split your data into training and validation sets:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, f1_score

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
tree_pred = tree.predict(X_val)

forest = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)
forest.fit(X_train, y_train)
forest_pred = forest.predict(X_val)

for name, predictions in [
    ("Decision Tree", tree_pred),
    ("Random Forest", forest_pred)
]:
    print(name)
    print("Accuracy:", accuracy_score(y_val, predictions))
    print("Macro F1:", f1_score(
        y_val, predictions, average="macro", zero_division=0
    ))
```

What the code does:

- **fit()** teaches the model using the training data.
 - **predict()** asks the model to predict poker hand categories for the validation data.
 - **accuracy_score()** measures the percentage of correct predictions.
 - **macro F1** calculates the F1-score for each class and gives each class equal weight. This matters because rare poker hands can be difficult to predict.
- The validation data should be held aside from training. Also, check the number of examples in each class before splitting. Some categories are extremely rare, which can make reliable evaluation difficult.

3. How do you compare the models?

Imagine your results look like this. These are illustrative numbers, not actual project results:

Model	Accuracy	Macro F1
Baseline	50.1%	0.067
Decision tree	75%	0.35
Random forest	85%	0.50

You would look at both metrics, along with the confusion matrix, to understand where each model succeeds and struggles.

The goal isn't just to get a high accuracy score. You also want to understand whether the model can recognize less common hands like full houses, four of a kind, and royal flushes.

In a nutshell

- **Baseline:** Establishes a simple starting score.

- **Decision tree:** Tests whether a single tree can learn useful patterns.
- **Random forest:** Tests whether combining many trees improves predictions.
- **Comparison:** Uses the same validation data and metrics to compare results fairly.